Lecture Slip, CS 235
Nov 3, 2026, Tuesday
Operators, Unique Strings, Trees

Name:	
EmpID:	

1. **Operator combinations.** The function `ops(v, k, expr)` from class fills every gap between the numbers in `v` with `+`, `-`, `*`, `/`.

   (a) How many lines does it print for `v = {1, 2, 3, 4}`?

   (b) After each recursive call, the code runs `expr.resize(len)`. Why not call `expr.pop_back()` twice instead?

   Answer: (a) Three gaps with four operators each: $4^3 = \mathbf{64}$ lines.

   (b) `std::to_string(v[k])` can add more than one character (for example `"12"`), so a fixed number of `pop_back()` calls would remove the wrong amount. Saving the length before the choose and restoring it afterwards works for any number.

2. **Printing unique strings.** `printUnique` keeps a `std::set<char> tried` inside each call, and runs `if (!tried.insert(s[i]).second) { continue; }`.

   (a) For `s = "ABB"` at the first call (`k = 0`), which values of `i` are skipped, and why?

   (b) How many lines do `printAll("ABB")` and `printUnique("ABB")` print?

   Answer: (a) `i = 0` inserts `'A'` (new, so it is explored). `i = 1` inserts `'B'` (new, so it is explored). `i = 2` is `'B'` again: `insert` fails, so `continue` skips the whole repeated branch. So $i = 2$ is skipped.

   (b) `printAll`: **6** lines (each of the 3 different strings twice). `printUnique`: **3** lines.

3. **Make 24, Task B.** In Task B each value is a pair (text, number).

   (a) `arithmetic` is given `{"2", 2}` and `{"5", 5}`. Write the two **subtraction** entries it produces (text and value), in the required order $a - b$, then $b - a$.

   (b) In `make24`, a recursive call returns the empty string. What does the caller do next? What does the caller do when the string is *not* empty?

   Answer: (a) `{"(2-5)", -3}` and `{"(5-2)", 3}`. The text keeps its parentheses; the value is computed.

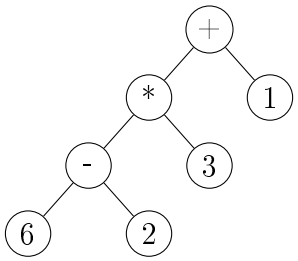
   (b) Empty means "no solution along this path": the caller undoes the choice (`cloneVec.pop_back()`) and tries the next candidate. A non-empty string is already a complete expression for 24, so the caller immediately returns it, and every caller above does the same.

4. **Expression trees.** Consider $(6 - 2) * 3 + 1$.

   (a) Build the tree: write the operations in the order they are performed, then draw the tree.

   (b) Write the **pre-order** and the **post-order** traversals.

   Answer: (a) Order: $6 - 2$, then $(6 - 2) * 3$, then $(6 - 2) * 3 + 1$. The last operation, `+`, is the root:



(b) Pre-order (root, left, right): `+ * - 6 2 3 1`. Post-order (left, right, root): `6 2 - 3 * 1 +`.

5. **Pace survey.** Please help me set the pace of the course. Circle one.

Too **slow** (I can go faster)	**Normal** (about right)	Too **fast** (please slow down)

If "too fast", which topic would you like me to revisit?

Answer: Tally the three circles and the topics requested; there is no correct answer. Only the counts matter.

6. Do you have any questions about today's class? If so, please provide details. Thank you.